The Blazor edition

How blazor.mvcprogrammer.com is built, how it differs from the vanilla edition at mvcprogrammer.com, and the rules for changing, testing, and deploying it.

1. Why there are two editions

The portfolio ships twice. The default edition is plain HTML, CSS, and JavaScript at the repository root, served at mvcprogrammer.com. The Blazor edition is a standalone Blazor WebAssembly app on .NET 10 under `blazor/`, served at blazor.mvcprogrammer.com. Both show the same content, load the same stylesheet, and call the same Experience API. The Blazor edition exists to show that content consumed from C# running in the browser, not to replace the static site.

The one rule that matters

Every content, markup, behavior, or styling change made to one edition is made to the other in the same commit. Before finishing any change to the site, check the other edition for the same change. Section 4 lists which files are shared by linking (one edit) and which are duplicated (two edits).

The deliberate exception

The linked tag in the About section's core skills differs on purpose. The default edition shows **Blazor** linking to blazor.mvcprogrammer.com; the Blazor edition shows **JavaScript** linking to mvcprogrammer.com. Each edition points at the other. Keep that asymmetry. A bUnit test, `AboutSection_ShouldLinkJavaScriptTagToStaticEdition`, pins the Blazor side of it.

The Blazor edition stays off the GitHub profile README until the owner decides otherwise.

2. Where things live

Both projects follow the folder-by-kind layout from the coding standards: plural folder names, a matching suffix on each class, and a namespace that follows the folder.

Path	What it holds
`blazor/src/Portfolio.Blazor/`	The app. One project, `Microsoft.NET.Sdk.BlazorWebAssembly`, net10.0, nullable and analyzers on, warnings are errors.
`Program.cs`	Host builder. Registers `App` and `HeadOutlet`, then calls `AddPortfolioServices()`.
`App.razor`	Router. No `FocusOnNavigate`, because in-page links are hash navigations.
`Layout/MainLayout.razor`	Skip link, icon sprite, header, `<main>`, footer.
`Pages/Home.razor`	The one page: a list of section components in page order.
`Sections/`	One component per section of the page: Hero, About, Services, Freelance, Portfolio, Quote, Resume, Contact, Colophon. Markup lifted from `index.html`.

Path	What it holds
`Components/`	Header (menu, scroll state, current section), footer, icon and icon sprite, social links, the experience timeline with its skill filter, and the kiosk demo dialog.
`Configurations/SiteOptions`	Lambda URLs and API paths, bound from `wwwroot/appsettings.json` and validated with data annotations.
`Interfaces/, Services/`	`IExperienceClient` and `IContactClient` (typed HttpClients) and `IContactTopicState` (the call-to-action buttons preselect the form topic).
`Models/, Requests/`	Records matching `api/openapi.yaml` and the contact Lambda's JSON body.
`ViewModels/ContactForm.cs`	Mutable, data-annotated model bound to the `EditForm`; converts itself to a `ContactRequest`.
`Serialization/`	`System.Text.Json` source-generator context, so the trimmed build needs no reflection.
`Extensions/`	`ServiceCollectionExtensions` (DI registration) and `PositionExtensions` (date and place labels).
`wwwroot/js/site.js`	The only JavaScript: document events, IntersectionObserver, matchMedia, the native dialog, and focus. Loaded as an ES module through `IJSRuntime`.
`wwwroot/blazor.css`	Boot screen and error bar only. Everything else is the shared `styles.css`.
`blazor/tests/Portfolio.Blazor.Tests/`	bUnit on xunit.v3. Section 6.
`blazor/README.md`	The short version of this document, kept in the repo.

The root `mvcprogrammer.com.sln` includes the Blazor app, its tests, the Experience API, its seed tool, and the API tests, so an IDE opens everything at once.

3. How the Blazor edition differs from the vanilla edition

The visible page is the same. The differences are in how it gets there.

Concern	Vanilla edition (root)	Blazor edition (blazor/)
Rendering	Hand-written HTML. The page reads fully with JavaScript disabled.	Razor components rendered by .NET on WebAssembly. A boot screen shows until the runtime loads; nothing renders without it.
Where state lives	DOM attributes and closures in `main.js`.	C# fields in the component code-behinds; the UI re-renders from them.
JavaScript	`main.js`, about 470 lines, owns every behavior.	`site.js`, about 100 lines, exposes only the browser APIs .NET cannot reach. Each watcher returns a handle with `dispose()`.

Concern	Vanilla edition (root)	Blazor edition (blazor/)
Experience timeline	The timeline is in the HTML. When `/api/experience` answers, the script re-renders it and shows the skill filter. On failure the HTML stays as it was.	`ExperienceTimeline` loads from the API. If that fails, times out, or returns HTML, it loads the bundled seed `data/positions.json` (linked from `api/seed/`). The filter shows only when skills load.
Skill chips	Same ordering: pinned list first, then the rest, then demoted; twelve shown before the more button.	Same constants and ordering in C#. Pinning a different skill means editing both.
Contact form	HTML5 validation plus `checkValidity()`; fetch to the Lambda; status text for 429, other failures, and success.	`EditForm` over `ContactForm` with data annotations and a conditional phone rule; `ContactClient` posts; the same status messages, plus a timeout message when the 20 second client timeout cancels.
Topic preselection	Call-to-action buttons set the select's value directly.	`IContactTopicState`, a singleton the buttons call and the form subscribes to.
Configuration	Endpoints sit in `data-` attributes in the HTML.	`SiteOptions` bound from `wwwroot/appsettings.json` and validated at startup.
Header behavior	Scroll listener, IntersectionObserver, Escape and outside-click handlers in `main.js`.	The same observers in `site.js`, reporting to `SiteHeader` through `[JSInvokable]` methods. Document listeners exist only while the menu is open.
Kiosk demo	Button opens a native dialog; the iframe is cleared on close.	`KioskDialog` renders the iframe only while open and tears it down on `OnClosed`.
Build output	The files as committed.	`dotnet publish`: trimmed, invariant globalization, no time zone data, no fingerprint placeholders (the import map would be an inline script, which the CSP forbids).
Hosting	Root of the private S3 bucket, distribution for mvcprogrammer.com.	`blazor/` prefix of the same bucket, distribution `E2NFN5MW02S8Y9`, own ACM certificate, CSP with `'wasm-unsafe-eval'`, S3 403 rewritten to `index.html` so deep links reach the router.
Tests	None in the repo for the static page.	111 bUnit and xUnit cases, run before every publish.

4. Shared by linking versus duplicated by hand

The Blazor project file pulls the shared assets from the repository root with `<Content Link=...>` items, so they are one file on disk and cannot drift. Everything else is written twice.

Edit once (linked into wwwroot)

- `styles.css`, the whole stylesheet.
- `assets/**`: fonts (Inter and JetBrains Mono as variable WOFF2 subsets) and images.
- `favicon.svg`, `favicon.ico`, `apple-touch-icon.png`.

- `api/seed/positions.json`, served to the Blazor edition as `data/positions.json`.

Edit twice (one copy per edition)

- Markup: `index.html` and again in `blazor/.../Sections/*.razor` and `Components/*.razor`.
- Behavior: `main.js` and again in the Razor code-behind files plus `wwwroot/js/site.js`.
- Constants that shape the page: pinned and demoted skills, the visible chip count, status messages, section ids and labels.
- Endpoints: `data-` attributes in `index.html` and `wwwroot/appsettings.json`.

When a change touches a duplicated file, the commit is not finished until both copies match. When the Blazor copy changes behavior, add or update the bUnit test that proves it.

5. Running it locally

Node is not needed. The .NET 10 SDK is. All commands run from the repository root.

```
dotnet run --project blazor/src/Portfolio.Blazor       # http://localhost:5115
dotnet publish blazor/src/Portfolio.Blazor -c Release -o publish
```

The dev server has no `/api`, so the timeline falls back to the bundled seed and the skill filter stays hidden. To see the filter, serve the publish output behind the real CloudFront behaviors or any host that proxies `/api` to the Experience API. The Experience API itself runs locally with `dotnet run --project api/src/Experience.Api` and serves the seed without AWS credentials.

The contact Lambda rejects Origins it does not know, so the form cannot send from a local preview in either edition.

6. Testing

The suite is bUnit 2 on xunit.v3 4, running on Microsoft.Testing.Platform. Components render against fakes of the two client interfaces and a bUnit JS module stub for `site.js`, so the skill filter, the seed fallback, the contact form's validation and status messages, the mobile menu, and the kiosk dialog are all exercised without a browser. The two HTTP clients are tested over a stubbed `HttpMessageHandler`. The deploy workflow runs the suite before every publish, so a red test blocks the Blazor deploy.

Commands

```
# run the suite
dotnet test --project blazor/tests/Portfolio.Blazor.Tests
# print every test name
dotnet test --project blazor/tests/Portfolio.Blazor.Tests --output Detailed
# discover without running
dotnet test --project blazor/tests/Portfolio.Blazor.Tests --list-tests
# one class, one method (a leading or trailing * is the wildcard)
dotnet test --project blazor/tests/Portfolio.Blazor.Tests --filter-class "*ContactSectionTests"
dotnet test --project blazor/tests/Portfolio.Blazor.Tests \
  --filter-method "*SendAsync_ShouldShowTimeoutMessageOnCancellation"
# the whole solution from the repository root: Blazor tests and API tests
dotnet test
```

The root `global.json` opts `dotnet test` into Microsoft.Testing.Platform, so the switches above are the platform's. `--filter-class` and `--filter-method` take a leading or trailing `*`. **VSTest-era options are not understood.** Passing `--logger "console;verbosity=normal"` makes the run report Zero tests ran and exit with code 5; that is the flag, not the tests.

Conventions the suite follows

- Test class is the class under test plus `Tests`; the project is `.Tests`. Method names are `Method_ShouldExpectedResult`, with `ShouldThrow`, `ShouldTransform`, and friends as the suffix.
- Arrange, Act, Assert, each marked with a comment. Scenario setup across several fakes lives in `Arrange_()` helpers; fixtures come from `Create()` factories.
- Regions group constructor tests first, then one region per method under test, then `Helper Methods` at the bottom.
- Every test has an XML summary stating the behavior it validates.
- Dependencies of the class under test are faked, never the class itself. `Assert.Equal` for exact values, `Assert.Single` when one element is expected, `Assert.ThrowsAsync` for async failures, `CancellationToken.None` unless cancellation is the subject.
- Async component assertions use `cut.WaitForAssertionAsync(...)` because the first render happens before `OnInitializedAsync` completes.

Test project layout

Folders mirror the app. The counts are test methods; theories expand to more cases at run time (111 in all).

File	Tests	What it proves
Components/ExperienceTimelineTests.cs	15	API render, date and place labels, every fallback path (error, non-JSON, timeout, empty), chip order and the more button, filtering, restore without refetch, cancellation on dispose.
Components/SiteHeaderTests.cs	10	Watchers start on render, menu toggle and document watch, close on nav click, focus return, scrolled class, current-section marking, module release.
Components/KioskDialogTests.cs	7	No iframe while closed, open shows the modal, open is idempotent, close removes the iframe, close button behavior, the portfolio card opens it.
Components/ChromeComponentTests.cs	4	Icon references a sprite symbol, every symbol exists, three social profiles, footer year and legal links.
Sections/ContactSectionTests.cs	12	Phone field hidden by default, topic preselection and unsubscribe, phone reveal, post and confirmations, honeypot forwarded, 429, generic failure, timeout, no post when invalid.
Sections/StaticSectionTests.cs	4	The JavaScript tag links to the static edition, the resume button uses the configured URL, the colophon links to the Blazor source, Home renders every section in order.
Sections/CallToActionTests.cs	2	The Services and Freelance buttons select their contact topic.

File	Tests	What it proves
Services/ExperienceClientTests.cs	10	Paths and deserialization, query-string escaping, blank skill ignored, error status, HTML body, null body, cancellation, skills, seed.
Services/ContactClientTests.cs	5	Null guards, camelCase JSON to the endpoint, status code on rejection, cancellation.
Services/ContactTopicStateTests.cs	6	Notify every subscriber, no subscribers, unsubscribe during notification, unsubscribe stops notifications, null guards.
Extensions/ServiceCollectionExtensionsTests.cs	4	Clients and state registered, options bound, options validated, bad arguments throw.
Extensions/PositionExtensionsTests.cs	5	Month labels, unparseable input returned as is, place label per work mode, null guards.
ViewModels/ContactFormTests.cs	5	Validation passes and fails as expected, phone required only for a call, the request drops the phone otherwise.
Fakes/, Helpers/		`FakeExperienceClient` and `FakeContactClient` (scriptable, record their calls), `StubHttpMessageHandler`, `TestData` factories, and `BunitContextExtensions` (options, services, the `site.js` module stub in loose mode).

7. Deploying

The `deploy-blazor` job in .github/workflows/deploy.yml runs on every push to `main`, in parallel with the static site's job. It sets up .NET 10, runs the tests in Release, publishes, assumes the deploy role through OIDC, and syncs the output in two passes before invalidating the distribution.

- Fingerprinted framework files under `_framework/` get a one-week browser TTL and `immutable`.
- Everything else, including the two unfingerprinted loaders `blazor.webassembly.js` and `dotnet.js`, gets a five-minute browser TTL.
- Pre-compressed `.br` and `.gz` twins stay out of the bucket; CloudFront compresses on the fly.

A manual deploy is the same two syncs plus an invalidation; the exact commands are in the Blazor section of docs/DEPLOY.md, together with the distribution, certificate, Route 53, bucket policy, and IAM details. Resource IDs are documented there once and are not repeated here.

8. Checklist for a change

Use this before marking any site change done.

Step	Do
1	Make the change in the edition you started with.
2	If the file is shared by linking (section 4), you are done with content. If it is duplicated, make the matching change in the other edition.
3	If the Blazor behavior changed, add or update the bUnit test that proves it. Follow the naming and layout conventions in section 6.
4	Run `dotnet test --project blazor/tests/Portfolio.Blazor.Tests` and make it green. Run `dotnet build` on the app; warnings are errors.
5	Preview both editions locally: `python3 -m http.server 8080` at the root and `dotnet run --project blazor/src/Portfolio.Blazor`.
6	Check the About tag asymmetry survived: Blazor on the static site, JavaScript on the Blazor site, each linking to the other.
7	Commit both editions together. No AI attribution trailers in the message.
8	Push to `main` only when asked. The workflow tests, publishes, and deploys both editions.

9. Links

What	Where
Live, Blazor edition	https://blazor.mvcprogrammer.com
Live, vanilla edition	https://mvcprogrammer.com
Repository	https://github.com/mvcprogrammer/mvcprogrammer.com
Blazor app source	blazor/src/Portfolio.Blazor
Blazor tests	blazor/tests/Portfolio.Blazor.Tests
Blazor README	blazor/README.md
Deploy walkthrough	docs/DEPLOY.md
Deploy workflow	.github/workflows/deploy.yml
Experience API spec	api/openapi.yaml
Experience API tests	api/tests/Experience.Api.Tests
Contact Lambda	infra/lambda/contact/index.mjs
Workflow runs	GitHub Actions